

cameras and smartphones, so the file names and directory names are not consistent. A convenient option is to use the date and time of an image from the metadata and rename files accordingly. An example code follows:

```
import os
import datetime
import pyexiv2
EXIF_DATE = 'Exif.Image.DateTime'
EXIF_ORIG_DATE = 'Exif.Photo.DateTimeOriginal'
def rename_file(p,f,fpref,ctr):
    fold,fext = f.rsplit('.',1) # separate the ext, e.g.
    jpg
    fname = fpref + "-%04i"%ctr # add a serial number to
    ensure uniqueness
    fnew = '.'.join((fname,fext))
    os.rename('.'.join((p,f)), '.'.join((p,fnew)))

def process_files(path, files):
    ctr = 0
    for f in files:
        try:
            metadata=pyexiv2.ImageMetadata('.'.
            join((path,f)))
            metadata.read()
            if EXIF_ORIG_DATE in metadata.exif_keys:
                timestamp = metadata[EXIF_ORIG_DATE].human_value
            else:
                timestamp = metadata[EXIF_DATE].human_value
            datepref = '_'.join([ x.replace(':', '-') for x in
            timestamp.split(' ')])
            rename_file(path,f,datepref,ctr)
            ctr += 1
        except:
            print('Error in %s/%s'%(path,f))
```

```
for path, dirs, files in os.walk('.'): # work with current
    directory for convenience
    if len(files) > 0:
        process_files(path, files)
```

All the file names now have a consistent file name. Since the photo managing software provides a way to view the photos by time, it seems that organising the files into directories that have meaningful names may be preferable. You can move photos into directories/albums that are meaningful. The photo management software will let you view photos either by albums or by dates.

Reducing clutter and duplicates

Over time, my collection included multiple copies of the same photos. In the old days, to share photos easily, I used to even keep low resolution copies. Digikam has an excellent option of identifying similar photos. However, each photo needs to be handled individually. A very convenient tool for finding the duplicate files and managing them programmatically is <http://www.jhnc.org/findimagedupes/>. The output of this program contains each set of duplicate files on a separate line.

You can use the Python Pillow and Matplotlib packages to display the images. Use the image's size to select the image with the highest resolution among the duplicates, retain that and delete the rest.

One thing is certain, though. After all the work is done, it is a pleasure to look at the photographs and relive all those old memories. **END** 

 By: Dr Anil Seth

The author has earned the right to do what interests him. You can find him online at <http://sethanil.com> and <http://sethanil.blogspot.com>.

OSFY Magazine Attractions During 2019-20

MONTH	THEME
March 2019	Mobile/Web App Development and Optimisation
April 2019	Cybersecurity, Open Source Firewall, Network Security and Monitoring
May 2019	AI, Deep Learning and Machine Learning
June 2019	DevOps Special
July 2019	Blockchain and Open Source
August 2019	Database Management and Optimisation
September 2019	Open Source and IoT
October 2019	Cloud Special: BigData, Hadoop, PaaS, SaaS, IaaS and Cloud
November 2019	Open Source on Windows
December 2019	Open Source Programming (Languages and Tools)
January 2020	Data Security, Storage and Backup
February 2020	Best in the World of Open Source (Tools and Services)